// 文件: App.vue

```
<template>
  <router-view />
</template>
<style>
/* 全局重置 */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
html,
body {
  width: 100%;
  height: 100%;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, 'Helvetica Neue',
    Arial, 'PingFang SC', 'Hiragino Sans GB', 'Microsoft YaHei', sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  -webkit-tap-highlight-color: transparent;
  overscroll-behavior: none;
}
#app {
  width: 100%;
  height: 100%;
}
/* Leaflet 标记自定义样式 */
.poi-marker {
  background: transparent !important;
  border: none !important;
  display: flex;
  flex-direction: column;
  align-items: center;
}
.poi-marker__dot {
  width: 12px;
  height: 12px;
  border-radius: 50%;
  border: 2px solid white;
  box-shadow: 0 1px 4px rgba(0, 0, 0, 0.3);
}
.poi-marker__label {
  margin-top: 2px;
  font-size: 14px;
  color: var(--color-text);
```

```
  background: rgba(255, 255, 255, 0.9);
  padding: 2px 8px;
  border-radius: 8px;
  white-space: nowrap;
  box-shadow: 0 1px 2px rgba(0, 0, 0, 0.1);
  font-weight: 500;
}
</style>
```

// 文件: components/common/AudioPlayer.vue

```
<template>
  <div v-if="currentPoi" class="audio-player">
    <!-- 播放信息 -->
    <div class="audio-player__info">
      <van-icon :name="isPlaying ? 'pause-circle' : 'play-circle'" class="audio-player__toggle"
@click="togglePlayback" />
      <div class="audio-player__text">
        <span class="audio-player__title">{{ currentPoi.name }}</span>
        <span class="audio-player__time">
          {{ formatTime(audioStore.currentTime) }} / {{ formatTime(audioStore.duration) }}
        </span>
      </div>
    </div>
    <!-- 进度条 -->
    <van-slider
      :model-value="progress"
      :max="100"
      bar-height="2px"
      active-color="var(--color-primary)"
      @change="handleSeek"
    />
    <!-- 关闭按钮 -->
    <van-icon name="cross" class="audio-player__close" @click="audioStore.stop()" />
  </div>
</template>
<script setup>
import { computed } from 'vue'
import { useAudio } from '@/composables/useAudio'
const { audioStore, togglePlayback, getCurrentPoi, formatTime } = useAudio()
const currentPoi = computed(() => getCurrentPoi())
const isPlaying = computed(() => audioStore.isPlaying)
const progress = computed(() => {
  if (!audioStore.duration) return 0
  return (audioStore.currentTime / audioStore.duration) * 100
})
function handleSeek(value) {
```

```
  const time = (value / 100) * audioStore.duration
  audioStore.seek(time)
}
</script>
<style scoped>
.audio-player {
  display: flex;
  align-items: center;
  gap: var(--spacing-sm);
  background: var(--color-white);
  padding: var(--spacing-sm) var(--spacing-lg);
  border-top: 1px solid var(--color-border);
}
.audio-player__info {
  display: flex;
  align-items: center;
  gap: var(--spacing-sm);
  min-width: 0;
}
.audio-player__toggle {
  font-size: 28px;
  color: var(--color-primary);
  cursor: pointer;
  flex-shrink: 0;
}
.audio-player__text {
  display: flex;
  flex-direction: column;
  min-width: 0;
}
.audio-player__title {
  font-size: 13px;
  font-weight: 500;
  color: var(--color-text);
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
.audio-player__time {
  font-size: 11px;
  color: var(--color-text-secondary);
}
.audio-player :deep(.van-slider) {
  flex: 1;
}
.audio-player__close {
```

```
  font-size: 16px;
  color: var(--color-text-secondary);
  cursor: pointer;
  flex-shrink: 0;
  padding: 4px;
}
</style>
```

// 文件: components/common/PoiBottomSheet.vue

```
<template>
  <van-popup
    v-model:show="visible"
    position="bottom"
    round
    :style="{ maxHeight: '45vh' }"
    @close="handleClose"
  >
    <div v-if="poi" class="poi-bottom-sheet">
      <!-- 拖拽指示条 -->
      <div class="poi-bottom-sheet__handle"></div>
      <!-- POI 基本信息 -->
      <div class="poi-bottom-sheet__header" @click="goDetail">
        <div class="poi-bottom-sheet__info">
          <h3 class="poi-bottom-sheet__name">{{ poi.name }}</h3>
          <p class="poi-bottom-sheet__desc">{{ poi.description }}</p>
          <div v-if="poi.tags?.length" class="poi-bottom-sheet__tags">
            <van-tag
              v-for="tag in poi.tags.slice(0, 3)"
              :key="tag"
              type="primary"
              plain
              size="medium"
            >
              {{ tag }}
            </van-tag>
          </div>
        </div>
        <van-icon name="arrow" class="poi-bottom-sheet__arrow" />
      </div>
      <!-- 快捷操作 -->
      <div class="poi-bottom-sheet__actions">
        <button
          v-if="poi.audioUrl"
          class="poi-bottom-sheet__action-btn"
          @click.stop="handlePlayAudio"
        >
```

```
        <van-icon :name="isPlaying ? 'pause-circle-o' : 'play-circle-o'" />
        <span>{{ isPlaying ? '暂停' : '语音讲解' }}</span>
      </button>
      <button class="poi-bottom-sheet__action-btn" @click.stop="handleNavigate">
        <van-icon name="guide-o" />
        <span>导航</span>
      </button>
      <button v-if="poi.phone" class="poi-bottom-sheet__action-btn" @click.stop="handleCall">
        <van-icon name="phone-o" />
        <span>电话</span>
      </button>
    </div>
  </div>
 </van-popup>
</template>
<script setup>
import { computed } from 'vue'
import { useRouter } from 'vue-router'
import { usePoiStore } from '@/stores/poi'
import { useAudio } from '@/composables/useAudio'
import { useGeolocation } from '@/composables/useGeolocation'
import { useMapStore } from '@/stores/map'
const router = useRouter()
const poiStore = usePoiStore()
const mapStore = useMapStore()
const { audioStore, playPoiAudio } = useAudio()
const { openNavigation, callPhone } = useGeolocation()
const poi = computed(() => poiStore.selectedPoi)
const visible = computed({
 get: () => !!poiStore.selectedPoi,
 set: () => poiStore.clearSelection(),
})
const isPlaying = computed(
 () => audioStore.isPlaying && audioStore.currentPoiId === poi.value?.id
)
function handleClose() {
 poiStore.clearSelection()
}
function goDetail() {
 if (!poi.value) return
 router.push(`/poi/${poi.value.id}`)
}
function handlePlayAudio() {
 if (!poi.value) return
 playPoiAudio(poi.value)
}
```

```
function handleNavigate() {
  if (!poi.value) return
  const [lat, lng] = mapStore.percentToLatLng(poi.value.x, poi.value.y)
  openNavigation([lat, lng], poi.value.name)
}
function handleCall() {
  if (!poi.value?.phone) return
  callPhone(poi.value.phone)
}
</script>
<style scoped>
.poi-bottom-sheet {
  padding: var(--spacing-md) var(--spacing-lg);
  padding-bottom: calc(var(--spacing-lg) + env(safe-area-inset-bottom));
}
.poi-bottom-sheet__handle {
  width: 32px;
  height: 4px;
  background: var(--color-border);
  border-radius: 2px;
  margin: 0 auto var(--spacing-md);
}
.poi-bottom-sheet__header {
  display: flex;
  align-items: flex-start;
  gap: var(--spacing-sm);
  cursor: pointer;
}
.poi-bottom-sheet__info {
  flex: 1;
}
.poi-bottom-sheet__name {
  font-size: 18px;
  font-weight: 600;
  color: var(--color-text);
  margin: 0 0 var(--spacing-xs);
}
.poi-bottom-sheet__desc {
  font-size: 13px;
  color: var(--color-text-secondary);
  margin: 0 0 var(--spacing-sm);
  line-height: 1.5;
  display: -webkit-box;
  -webkit-line-clamp: 2;
  -webkit-box-orient: vertical;
  overflow: hidden;
```

```
}
.poi-bottom-sheet__tags {
  display: flex;
  gap: var(--spacing-xs);
  flex-wrap: wrap;
}
.poi-bottom-sheet__arrow {
  color: var(--color-text-secondary);
  margin-top: 4px;
}
.poi-bottom-sheet__actions {
  display: flex;
  gap: var(--spacing-md);
  margin-top: var(--spacing-lg);
  padding-top: var(--spacing-md);
  border-top: 1px solid var(--color-border);
}
.poi-bottom-sheet__action-btn {
  flex: 1;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 4px;
  padding: var(--spacing-sm);
  background: var(--color-bg);
  border: none;
  border-radius: var(--radius-md);
  color: var(--color-primary);
  font-size: 12px;
  cursor: pointer;
}
.poi-bottom-sheet__action-btn:active {
  background: var(--color-border);
}
.poi-bottom-sheet__action-btn .van-icon {
  font-size: 20px;
}
</style>
```

// 文件: components/common/SearchBar.vue

```
<template>
  <div class="search-bar" @click="$emit('click')">
    <van-icon name="search" class="search-bar__icon" />
    <span class="search-bar__placeholder">{{ placeholder }}</span>
  </div>
</template>
```

```
<script setup>
defineProps({
  placeholder: {
    type: String,
    default: '搜索景点、美食、住宿...',
  },
})
defineEmits(['click'])
</script>
<style scoped>
.search-bar {
  display: flex;
  align-items: center;
  gap: var(--spacing-sm);
  background: var(--color-white);
  border-radius: var(--radius-round);
  padding: var(--spacing-sm) var(--spacing-lg);
  box-shadow: var(--shadow-sm);
  cursor: pointer;
  flex: 1;
  max-width: 280px;
  height: 36px;
}
.search-bar__icon {
  color: var(--color-text-secondary);
  font-size: 16px;
}
.search-bar__placeholder {
  color: var(--color-text-secondary);
  font-size: 13px;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
</style>
```

// 文件: components/common/VisitorCounter.vue

```
<template>
  <div class="visitor-counter">
    <van-icon name="friends-o" class="visitor-counter__icon" />
    <span class="visitor-counter__text">第 <strong>{{ displayCount }}</strong> 位访客</span>
  </div>
</template>
<script setup>
import { ref, onMounted } from 'vue'
const props = defineProps({
```

```
  count: {
    type: Number,
    default: 425,
  },
})
const displayCount = ref(0)
// 数字递增动画
onMounted(() => {
  const target = props.count
  const duration = 1500
  const step = Math.ceil(target / (duration / 16))
  const timer = setInterval(() => {
    displayCount.value = Math.min(displayCount.value + step, target)
    if (displayCount.value >= target) {
      clearInterval(timer)
    }
  }, 16)
})
</script>
<style scoped>
.visitor-counter {
  display: flex;
  align-items: center;
  gap: 4px;
  background: rgba(255, 255, 255, 0.9);
  border-radius: var(--radius-round);
  padding: 4px 10px;
  font-size: 12px;
  color: var(--color-text-secondary);
  box-shadow: var(--shadow-sm);
  backdrop-filter: blur(4px);
}
.visitor-counter__icon {
  font-size: 14px;
  color: var(--color-primary);
}
.visitor-counter__text strong {
  color: var(--color-primary);
  font-weight: 600;
}
</style>
```

// 文件: components/detail/ActionButtons.vue

```
<template>
  <div class="action-buttons">
    <!-- 路线导航 -->
```

```
    <button class="action-buttons__btn action-buttons__btn--primary" @click="handleNavigate">
      <van-icon name="guide-o" size="20" />
      <span>路线导航</span>
    </button>
    <!-- 电话拨打 -->
    <button
      v-if="poi?.phone"
      class="action-buttons__btn action-buttons__btn--secondary"
      @click="handleCall"
    >
      <van-icon name="phone-o" size="20" />
      <span>电话咨询</span>
    </button>
  </div>
</template>
<script setup>
import { showToast } from 'vant'
const props = defineProps({
  poi: {
    type: Object,
    required: true,
  },
})
function handleNavigate() {
  showToast('手绘地图模式下暂不支持路线导航')
}
function handleCall() {
  if (props.poi.phone) {
    window.location.href = `tel:${props.poi.phone}`
  }
}
</script>
<style scoped>
.action-buttons {
  display: flex;
  gap: var(--spacing-md);
  padding: var(--spacing-lg);
}
.action-buttons__btn {
  flex: 1;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: var(--spacing-sm);
  padding: var(--spacing-md);
  border-radius: var(--radius-round);
```

```
  font-size: 15px;
  font-weight: 500;
  border: none;
  cursor: pointer;
  transition: opacity 0.15s;
}
.action-buttons__btn:active {
  opacity: 0.8;
}
.action-buttons__btn--primary {
  background: var(--color-primary);
  color: white;
}
.action-buttons__btn--secondary {
  background: var(--color-white);
  color: var(--color-primary);
  border: 1px solid var(--color-primary);
}
</style>
```

// 文件: components/detail/PoiInfo.vue

```
<template>
  <div v-if="poi" class="poi-info">
    <!-- 封面图 -->
    <div class="poi-info__cover">
      <img
        v-if="poi.image"
        :src="poi.image"
        :alt="poi.name"
        class="poi-info__image"
      />
      <div v-else class="poi-info__placeholder">
        <van-icon name="photo-o" size="48" color="#ccc" />
        <span>暂无图片</span>
      </div>
    </div>
    <!-- 基本信息 -->
    <div class="poi-info__content">
      <h1 class="poi-info__name">{{ poi.name }}</h1>
      <!-- 标签 -->
      <div v-if="poi.tags?.length" class="poi-info__tags">
        <van-tag v-for="tag in poi.tags" :key="tag" type="primary" plain>
          {{ tag }}
        </van-tag>
      </div>
      <!-- 描述 -->
```

```
      <p class="poi-info__desc">{{ poi.description }}</p>
      <!-- 信息列表 -->
      <div class="poi-info__meta">
        <div v-if="poi.address" class="poi-info__meta-item">
          <van-icon name="location-o" />
          <span>{{ poi.address }}</span>
        </div>
        <div v-if="poi.openTime" class="poi-info__meta-item">
          <van-icon name="clock-o" />
          <span>{{ poi.openTime }}</span>
        </div>
        <div v-if="poi.phone" class="poi-info__meta-item">
          <van-icon name="phone-o" />
          <span>{{ poi.phone }}</span>
        </div>
      </div>
    </div>
  </div>
</template>
<script setup>
defineProps({
  poi: {
    type: Object,
    required: true,
  },
})
</script>
<style scoped>
.poi-info__cover {
  width: 100%;
  height: 200px;
  overflow: hidden;
  background: var(--color-bg);
}
.poi-info__image {
  width: 100%;
  height: 100%;
  object-fit: cover;
}
.poi-info__placeholder {
  width: 100%;
  height: 100%;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
```

```
  gap: var(--spacing-sm);
  color: var(--color-text-secondary);
  font-size: 13px;
}
.poi-info__content {
  padding: var(--spacing-lg);
}
.poi-info__name {
  font-size: 20px;
  font-weight: 600;
  color: var(--color-text);
  margin: 0 0 var(--spacing-sm);
}
.poi-info__tags {
  display: flex;
  gap: var(--spacing-xs);
  flex-wrap: wrap;
  margin-bottom: var(--spacing-md);
}
.poi-info__desc {
  font-size: 14px;
  line-height: 1.8;
  color: var(--color-text);
  margin: 0 0 var(--spacing-lg);
}
.poi-info__meta {
  display: flex;
  flex-direction: column;
  gap: var(--spacing-sm);
  padding-top: var(--spacing-md);
  border-top: 1px solid var(--color-border);
}
.poi-info__meta-item {
  display: flex;
  align-items: center;
  gap: var(--spacing-sm);
  font-size: 13px;
  color: var(--color-text-secondary);
}
.poi-info__meta-item .van-icon {
  color: var(--color-primary);
}
</style>
```

// 文件: components/detail/VideoPlayer.vue

```
<template>
```

```
  <div class="video-player">
    <video
      v-if="videoUrl"
      ref="videoRef"
      :src="videoUrl"
      :poster="poster"
      class="video-player__video"
      controls
      playsinline
      webkit-playsinline
      preload="metadata"
    >
      您的浏览器不支持视频播放
    </video>
    <div v-else class="video-player__placeholder">
      <van-icon name="video-o" size="48" color="#ccc" />
      <span>暂无视频</span>
    </div>
  </div>
</template>
<script setup>
import { ref } from 'vue'
defineProps({
  videoUrl: {
    type: String,
    default: '',
  },
  poster: {
    type: String,
    default: '',
  },
})
const videoRef = ref(null)
</script>
<style scoped>
.video-player {
  width: 100%;
  background: #000;
  border-radius: var(--radius-md);
  overflow: hidden;
}
.video-player__video {
  width: 100%;
  display: block;
  max-height: 240px;
}
```

```
.video-player__placeholder {
  width: 100%;
  height: 160px;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: var(--spacing-sm);
  color: var(--color-text-secondary);
  font-size: 13px;
  background: var(--color-bg);
}
</style>
```

// 文件: components/detail/VREntry.vue

```
<template>
  <div class="vr-entry" @click="handleClick">
    <div class="vr-entry__icon">
      <van-icon name="video-o" size="32" />
    </div>
    <div class="vr-entry__info">
      <span class="vr-entry__title">VR 全景</span>
      <span class="vr-entry__desc">沉浸式 360° 全景体验</span>
    </div>
    <van-icon name="arrow" class="vr-entry__arrow" />
  </div>
</template>
<script setup>
import { useRouter } from 'vue-router'
const router = useRouter()
const props = defineProps({
  poiId: {
    type: [String, Number],
    default: '',
  },
  poiName: {
    type: String,
    default: '',
  },
})
function handleClick() {
  router.push({
    path: '/panorama',
    query: {
      poiId: props.poiId,
      poiName: props.poiName,
```

```
    },
  })
}
</script>
<style scoped>
.vr-entry {
  display: flex;
  align-items: center;
  gap: var(--spacing-md);
  padding: var(--spacing-lg);
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  border-radius: var(--radius-lg);
  margin: var(--spacing-lg);
  cursor: pointer;
  color: white;
}
.vr-entry:active {
  opacity: 0.9;
}
.vr-entry__icon {
  width: 48px;
  height: 48px;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.2);
  display: flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
}
.vr-entry__info {
  flex: 1;
  display: flex;
  flex-direction: column;
  gap: 2px;
}
.vr-entry__title {
  font-size: 16px;
  font-weight: 600;
}
.vr-entry__desc {
  font-size: 12px;
  opacity: 0.8;
}
.vr-entry__arrow {
  opacity: 0.6;
}
```

```
</style>
```

// 文件: components/map/CategoryTabs.vue

```
<template>
  <div class="category-tabs">
    <div
      v-for="cat in poiStore.categories"
      :key="cat.key"
      class="category-tabs__item"
      :class="{ 'category-tabs__item--active': poiStore.activeCategory === cat.key }"
      @click="poiStore.setCategory(cat.key)"
    >
      <van-icon :name="cat.icon" class="category-tabs__icon" />
      <span class="category-tabs__label">{{ cat.label }}</span>
    </div>
  </div>
</template>
<script setup>
import { usePoiStore } from '@/stores/poi'
const poiStore = usePoiStore()
</script>
<style scoped>
.category-tabs {
  display: flex;
  background: var(--color-white);
  border-top: 1px solid var(--color-border);
  padding: var(--spacing-sm) 0;
  padding-bottom: calc(var(--spacing-sm) + env(safe-area-inset-bottom));
  overflow-x: auto;
  -webkit-overflow-scrolling: touch;
}
.category-tabs__item {
  flex: 1;
  min-width: 64px;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 4px;
  padding: var(--spacing-xs) var(--spacing-sm);
  cursor: pointer;
  transition: color 0.2s;
  color: var(--color-text-secondary);
}
.category-tabs__item--active {
  color: var(--color-primary);
}
```

```
.category-tabs__icon {
  font-size: 20px;
}
.category-tabs__label {
  font-size: 11px;
  white-space: nowrap;
}
</style>
```

// 文件: components/map/LeafletMap.vue

```
<template>
  <div ref="mapContainer" class="leaflet-map"></div>
</template>
<script setup>
import { ref, watch, onMounted } from 'vue'
import L from 'leaflet'
import { useMap } from '@/composables/useMap'
import { usePoiStore } from '@/stores/poi'
import { useMapStore } from '@/stores/map'
const props = defineProps({
  editMode: { type: Boolean, default: false },
})
const emit = defineEmits(['add-poi'])
const mapContainer = ref(null)
const { map, isReady, getMap } = useMap(mapContainer)
const poiStore = usePoiStore()
const mapStore = useMapStore()
// 存储地图上的 marker 实例
const markers = ref([])
// 地图就绪后，初始化标记
onMounted(() => {
  // isReady 变为 true 后由 watch 处理
})
// 监听过滤后的 POI 变化，更新地图标记
watch(
  () => poiStore.filteredPois,
  (pois) => {
    if (!isReady.value) return
    updateMarkers(pois)
  },
  { immediate: true }
)
// 等待地图就绪
watch(isReady, (ready) => {
  if (ready) {
    updateMarkers(poiStore.filteredPois)
```

```
  }
})
/**
 * 测量文字宽度（用于自适应标记容器）
 */
function measureTextWidth(text, font) {
  const canvas = measureTextWidth._canvas || (measureTextWidth._canvas = document.createElement('canvas'))
  const ctx = canvas.getContext('2d')
  ctx.font = font
  return Math.ceil(ctx.measureText(text).width)
}
/**
 * 更新地图上的 POI 标记
 */
function updateMarkers(pois) {
  // 清除旧标记
  markers.value.forEach((m) => m.remove())
  markers.value = []
  if (!map.value) return
  pois.forEach((poi) => {
    // 百分比 → 像素坐标 [pixelY, pixelX]
    const [pixelY, pixelX] = mapStore.percentToLatLng(poi.x, poi.y)
    // 根据文字长度自适应容器宽度
    const labelWidth = measureTextWidth(poi.name, '500 14px -apple-system, sans-serif')
    const containerWidth = Math.max(labelWidth + 16, 20) // 左右各 8px padding
    const containerHeight = 36
    // 创建自定义 HTML 标记
    const icon = L.divIcon({
      className: 'poi-marker',
      html: `<div class="poi-marker__dot" style="background:${getCategoryColor(poi.category)}"></div>
            <div class="poi-marker__label">${poi.name}</div>`,
      iconSize: [containerWidth, containerHeight],
      iconAnchor: [containerWidth / 2, 18],
    })
    const marker = L.marker([pixelY, pixelX], { icon }).addTo(map.value)
    marker.on('click', (e) => {
      e.originalEvent.stopPropagation()
      poiStore.selectPoi(poi)
    })
    markers.value.push(marker)
  })
}
/**
 * 获取分类对应的颜色
 */
function getCategoryColor(category) {
```

```
  const cat = poiStore.categories.find((c) => c.key === category)
  return cat ? cat.color : '#1989fa'
}
/**
 * 地图点击处理（编辑模式下添加 POI）
 */
function onMapClick(e) {
  if (!props.editMode) return
  // CRS.Simple: e.latlng 为像素坐标 [pixelY, pixelX]
  const { lat: pixelY, lng: pixelX } = e.latlng
  const { x, y } = mapStore.pixelToPercent(pixelY, pixelX)
  emit('add-poi', { x, y })
}
// 监听编辑模式切换，绑定/解绑地图点击
watch(
  () => props.editMode,
  (mode) => {
    if (!isReady.value || !map.value) return
    if (mode) {
      map.value.on('click', onMapClick)
    } else {
      map.value.off('click', onMapClick)
    }
  }
)
// 暴露方法供父组件调用
defineExpose({ getMap })
</script>
<style scoped>
.leaflet-map {
  width: 100%;
  height: 100%;
}
</style>
```

// 文件: components/map/MapControls.vue

```
<template>
  <div class="map-controls">
    <button class="map-controls__btn" @click="$emit('zoomIn')" aria-label="放大">
      <van-icon name="plus" />
    </button>
    <button class="map-controls__btn" @click="$emit('zoomOut')" aria-label="缩小">
      <van-icon name="minus" />
    </button>
  </div>
</template>
```

```
<script setup>
defineEmits(['zoomIn', 'zoomOut'])
</script>
<style scoped>
.map-controls {
  display: flex;
  flex-direction: column;
  gap: 1px;
  border-radius: var(--radius-md);
  overflow: hidden;
  box-shadow: var(--shadow-md);
}
.map-controls__btn {
  width: 40px;
  height: 40px;
  background: var(--color-white);
  border: none;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 18px;
  color: var(--color-text);
  cursor: pointer;
  transition: background 0.15s;
}
.map-controls__btn:active {
  background: var(--color-bg);
}
.map-controls__btn + .map-controls__btn {
  border-top: 1px solid var(--color-border);
}
</style>
```

// 文件: components/map/PoiEditDialog.vue

```
<template>
  <van-popup
    :show="show"
    position="bottom"
    round
    :style="{ maxHeight: '70vh' }"
    @update:show="$emit('update:show', $event)"
  >
    <div class="poi-edit-dialog">
      <van-cell-group inset title="新增 POI">
        <van-field
          v-model="form.name"
```

```
      label="名称"
      placeholder="请输入 POI 名称"
      required
      :rules="[{ required: true, message: '名称不能为空' }]"
    />
    <van-field
      v-model="categoryText"
      is-link
      readonly
      label="分类"
      placeholder="请选择分类"
      required
      @click="showCategoryPicker = true"
    />
    <van-field
      v-model="form.description"
      label="描述"
      type="textarea"
      placeholder="请输入描述（选填）"
      autosize
    />
    <van-field
      v-model="form.address"
      label="地址"
      placeholder="请输入地址（选填）"
    />
    <van-field
      v-model="form.phone"
      label="电话"
      type="tel"
      placeholder="请输入电话（选填）"
    />
    <van-field
      v-model="form.tags"
      label="标签"
      placeholder="多个标签用逗号分隔（选填）"
    />
  </van-cell-group>
  <div class="poi-edit-dialog__actions">
    <van-button block plain type="default" @click="onCancel">取消</van-button>
    <van-button block type="primary" @click="onConfirm">确定</van-button>
  </div>
</div>
<!-- 分类选择器 -->
<van-popup v-model:show="showCategoryPicker" position="bottom" round>
  <van-picker
```

```
        :columns="categoryOptions"
        @confirm="onCategoryConfirm"
        @cancel="showCategoryPicker = false"
      />
    </van-popup>
  </van-popup>
</template>
<script setup>
import { ref, reactive, computed, watch } from 'vue'
import { usePoiStore } from '@/stores/poi'
const props = defineProps({
  show: { type: Boolean, default: false },
  x: { type: Number, default: 0 },
  y: { type: Number, default: 0 },
})
const emit = defineEmits(['update:show', 'confirm'])
const poiStore = usePoiStore()
const form = reactive({
  name: '',
  category: '',
  description: '',
  address: '',
  phone: '',
  tags: '',
})
const showCategoryPicker = ref(false)
// 分类选项（排除 'all'）
const categoryOptions = computed(() =>
  poiStore.categories
    .filter((c) => c.key !== 'all')
    .map((c) => ({ text: c.label, value: c.key }))
)
// 当前选中分类的显示文本
const categoryText = computed(() => {
  const cat = poiStore.categories.find((c) => c.key === form.category)
  return cat ? cat.label : ''
})
// 弹窗打开时重置表单
watch(
  () => props.show,
  (val) => {
    if (val) {
      form.name = ''
      form.category = ''
      form.description = ''
      form.address = ''
```

```
      form.phone = ''
      form.tags = ''
    }
  }
)
function onCategoryConfirm({ selectedOptions }) {
  form.category = selectedOptions[0]?.value || ''
  showCategoryPicker.value = false
}
function onConfirm() {
  if (!form.name.trim()) return
  const tags = form.tags
    ? form.tags
        .split(/[,，]/)
        .map((t) => t.trim())
        .filter(Boolean)
    : []
  emit('confirm', {
    id: `poi-${Date.now()}`,
    name: form.name.trim(),
    category: form.category,
    description: form.description.trim(),
    x: props.x,
    y: props.y,
    image: '',
    audioUrl: '',
    audioDuration: 0,
    videoUrl: '',
    vrUrl: '',
    phone: form.phone.trim(),
    address: form.address.trim(),
    openTime: '',
    tags,
  })
}
function onCancel() {
  emit('update:show', false)
}
</script>
<style scoped>
.poi-edit-dialog {
  padding: var(--spacing-md) 0;
  padding-bottom: calc(var(--spacing-md) + env(safe-area-inset-bottom));
}
.poi-edit-dialog__actions {
  display: flex;
```

```
  gap: var(--spacing-md);
  padding: var(--spacing-lg) var(--spacing-lg) 0;
}
</style>
```

// 文件: components/map/PoiMarker.vue

```
<template>
  <div class="poi-marker-dot" :style="{ borderColor: color }">
    <span class="poi-marker-dot__inner" :style="{ background: color }"></span>
  </div>
</template>
<script setup>
defineProps({
  color: {
    type: String,
    default: '#1989fa',
  },
})
</script>
<style scoped>
.poi-marker-dot {
  width: 16px;
  height: 16px;
  border-radius: 50%;
  border: 2px solid;
  background: white;
  display: flex;
  align-items: center;
  justify-content: center;
  box-shadow: 0 1px 4px rgba(0, 0, 0, 0.2);
}
.poi-marker-dot__inner {
  width: 6px;
  height: 6px;
  border-radius: 50%;
}
</style>
```

// 文件: composables/useAudio.js

```
import { useAudioStore } from '@/stores/audio'
import { usePoiStore } from '@/stores/poi'
/**
 * 语音播放 composable
 * 封装语音播放的便捷方法
 */
export function useAudio() {
  const audioStore = useAudioStore()
```

```
  const poiStore = usePoiStore()
  /**
   * 播放指定 POI 的语音讲解
   * @param {object} poi - POI 数据对象
   */
  function playPoiAudio(poi) {
    if (!poi.audioUrl) return
    audioStore.play(poi.id, poi.audioUrl)
  }
  /**
   * 切换播放/暂停
   */
  function togglePlayback() {
    if (audioStore.isPlaying) {
      audioStore.pause()
    } else {
      audioStore.resume()
    }
  }
  /**
   * 获取当前正在播放的 POI 数据
   */
  function getCurrentPoi() {
    if (!audioStore.currentPoiId) return null
    return poiStore.getPoiById(audioStore.currentPoiId)
  }
  /**
   * 格式化时间为 mm:ss
   * @param {number} seconds
   */
  function formatTime(seconds) {
    if (!seconds || isNaN(seconds)) return '00:00'
    const m = Math.floor(seconds / 60)
    const s = Math.floor(seconds % 60)
    return `${String(m).padStart(2, '0')}:${String(s).padStart(2, '0')}`
  }
  return {
    audioStore,
    playPoiAudio,
    togglePlayback,
    getCurrentPoi,
    formatTime,
  }
}
```

// 文件: composables/useGeolocation.js

```
import { ref } from 'vue'
/**
 * 地理定位 composable
 * 用于获取用户当前位置，支持路线导航
 */
export function useGeolocation() {
  const location = ref(null)
  const error = ref(null)
  const loading = ref(false)
  /**
   * 获取用户当前位置
   */
  function getCurrentPosition() {
    loading.value = true
    error.value = null
    if (!navigator.geolocation) {
      error.value = '当前浏览器不支持地理定位'
      loading.value = false
      return
    }
    navigator.geolocation.getCurrentPosition(
      (pos) => {
        location.value = {
          lat: pos.coords.latitude,
          lng: pos.coords.longitude,
          accuracy: pos.coords.accuracy,
        }
        loading.value = false
      },
      (err) => {
        error.value = err.message
        loading.value = false
      },
      {
        enableHighAccuracy: true,
        timeout: 10000,
        maximumAge: 60000,
      }
    )
  }
  /**
   * 调起手机原生地图进行路线导航
   * 自动检测平台并选择合适的 URL scheme
   * @param {[number, number]} dest - [lat, lng] 目的地坐标
   * @param {string} destName - 目的地名称
   */
```

```
  function openNavigation(dest, destName) {
    const [lat, lng] = dest
    const name = encodeURIComponent(destName)
    // 尝试打开高德地图 App，失败则打开百度地图，最终 fallback 到系统地图
    const amapUrl = `iosamap://navi?sourceApplication=funmap&lat=${lat}&lon=${lng}&dev=0&style=2`
    const baiduUrl =
`baidumap://map/navi?query=${name}&destination=${name}&src=funmap&coord_type=gcj02`
    const appleUrl = `maps://maps.apple.com/?daddr=${lat},${lng}&dirflg=d`
    // 优先尝试 Apple Maps（iOS 原生）
    window.open(appleUrl, '_blank')
  }
  /**
   * 拨打电话号码
   * @param {string} phone
   */
  function callPhone(phone) {
    if (!phone) return
    window.open(`tel:${phone}`, '_self')
  }
  return {
    location,
    error,
    loading,
    getCurrentPosition,
    openNavigation,
    callPhone,
  }
}
```

// 文件: composables/useMap.js

```
import { ref, onMounted, onUnmounted } from 'vue'
import L from 'leaflet'
import { useMapStore } from '@/stores/map'
/**
 * Leaflet 地图 composable（CRS.Simple 像素坐标系）
 * 封装地图初始化、缩放控制等逻辑
 */
export function useMap(containerRef) {
  const mapStore = useMapStore()
  const map = ref(null)
  const isReady = ref(false)
  onMounted(async () => {
    if (!containerRef.value) return
    // 先加载配置
    await mapStore.loadConfig()
    // 自定义 CRS：scale = 256 / paddedDim，无 y-flip
```

```
// pixelX = scale * 2^z * crsX, pixelY = scale * 2^z * crsY
const paddedDim = mapStore.paddedDim
const scale = 256 / paddedDim
const crs = L.Util.extend({}, L.CRS.Simple, {
  // 覆盖 projection bounds 为实际图片尺寸（而非 LonLat 的 [-180,-90]~[180,90]）
  projection: L.Util.extend({}, L.CRS.Simple.projection, {
    bounds: new L.Bounds([0, 0], [paddedDim, paddedDim]),
  }),
  transformation: new L.Transformation(scale, 0, scale, 0), // 无 y-flip
  infinite: false,
})
map.value = L.map(containerRef.value, {
  crs,
  minZoom: 0,
  maxZoom: mapStore.maxZoom + 4,
  zoomSnap: 0, // 允许无级缩放（非整数级别）
  zoomDelta: 0.5, // 按钮每次缩放 0.5 级
  wheelPxPerZoomLevel: 60, // 滚轮缩放灵敏度（值越小越灵敏）
  touchZoom: false, // 禁用内置 touchZoom，用自定义 handler 替代
  bounceAtZoomLimits: false, // 到达缩放极限时不反弹
  zoomControl: false,
  attributionControl: false,
})
// 使用 tileLayer 显示手绘地图瓦片
// 自定义 getTileUrl：翻转 y 坐标以匹配瓦片生成时的 y-flip
const maxZoom = mapStore.maxZoom + 4
const tilesPath = mapStore.tilesPath
const tileLayer = L.tileLayer(`${tilesPath}/{z}/{x}/{y}.png`, {
  minZoom: 0,
  maxZoom,
  maxNativeZoom: mapStore.maxZoom, // 瓦片只生成到 maxZoom，超出级别用 CSS 放大
  noWrap: true,
  tileSize: 256,
})
// 覆盖 getTileUrl，翻转 y 坐标
const originalGetTileUrl = tileLayer.getTileUrl.bind(tileLayer)
tileLayer.getTileUrl = function (coords) {
  // coords.y 是 Leaflet 计算的 tile y（基于无 y-flip CRS）
  // 需要翻转为瓦片文件中的 y（ty=0 在图片顶部）
  const tilesPerSide = Math.pow(2, coords.z)
  const flippedY = tilesPerSide - 1 - coords.y
  return `${tilesPath}/${coords.z}/${coords.x}/${flippedY}.png`
}
tileLayer.addTo(map.value)
// === 自定义双指缩放（解决 Leaflet Drag 拦截问题）===
// 问题：Leaflet Drag handler 在 document 上注册 touchmove（bubble phase），
```

```
  //      会调用 preventDefault() 阻止我们的容器 listener 收到事件。
  // 解决：使用 capture phase 监听器，它在 bubble phase 之前触发，优先级更高。
  const mapEl = containerRef.value
  let pinchStartDist = 0
  let pinchDecided = false
  const PINCH_THRESHOLD = 1.15 // 手指张合超过 15% 触发
  function getTouchDist(e) {
    const t = e.touches
    const dx = t[0].clientX - t[1].clientX
    const dy = t[0].clientY - t[1].clientY
    return Math.sqrt(dx * dx + dy * dy)
  }
  // capture phase：优先于 document 上的 bubble phase listener
  mapEl.addEventListener('touchstart', (e) => {
    console.log('[PinchDebug] touchstart, touches:', e.touches.length)
    if (e.touches.length === 2) {
      pinchStartDist = getTouchDist(e)
      pinchDecided = false
      console.log('[PinchDebug] 2-finger detected, startDist:', pinchStartDist.toFixed(1))
      // 禁用 drag 防止它拦截 pinch 手势
      map.value.dragging.disable()
    }
  }, { passive: true, capture: true })
  mapEl.addEventListener('touchmove', (e) => {
    console.log('[PinchDebug] touchmove, touches:', e.touches.length, 'pinchStartDist:', pinchStartDist,
'decided:', pinchDecided)
    if (e.touches.length !== 2 || !pinchStartDist || pinchDecided) return
    e.preventDefault() // 阻止 Leaflet Drag 处理
    const dist = getTouchDist(e)
    const ratio = dist / pinchStartDist
    console.log('[PinchDebug] dist:', dist.toFixed(1), 'ratio:', ratio.toFixed(3))
    if (ratio > PINCH_THRESHOLD) {
      pinchDecided = true
      console.log('[PinchDebug] zoomIn!')
      map.value.zoomIn(1, { animate: true })
    } else if (ratio < 1 / PINCH_THRESHOLD) {
      pinchDecided = true
      console.log('[PinchDebug] zoomOut!')
      map.value.zoomOut(1, { animate: true })
    }
  }, { passive: false, capture: true })
  mapEl.addEventListener('touchend', (e) => {
    console.log('[PinchDebug] touchend, touches:', e.touches.length)
    if (e.touches.length < 2) {
      pinchStartDist = 0
      pinchDecided = false
```

```
        // 恢复 drag
        map.value.dragging.enable()
      }
    }, { passive: true, capture: true })
    // 调试信息：监听缩放事件，输出当前缩放级别和瓦片状态
    map.value.on('zoomend', () => {
      const z = map.value.getZoom()
      const tiles = containerRef.value.querySelectorAll('.leaflet-tile')
      const loaded = Array.from(tiles).filter(t => t.complete && t.naturalWidth > 0).length
      const failed = Array.from(tiles).filter(t => t.complete && t.naturalWidth === 0).length
      console.log(`[MapDebug] zoom=${z}, maxZoom=${maxZoom}, maxNativeZoom=${mapStore.maxZoom},
tiles: ${tiles.length} total / ${loaded} loaded / ${failed} failed`)
    })
    // 恢复上次地图状态（路由返回时），否则使用默认初始视图
    if (mapStore.lastZoom != null && mapStore.lastCenter != null) {
      map.value.setView(mapStore.lastCenter, mapStore.lastZoom)
      console.log(`[MapDebug] 恢复上次视图: zoom=${mapStore.lastZoom}, center=${mapStore.lastCenter}`)
    } else {
      map.value.setView(
        [mapStore.imgHeight / 2, mapStore.imgWidth / 2],
        mapStore.initialZoom
      )
    }
    // 监听用户操作，实时保存地图状态
    map.value.on('moveend zoomend', () => {
      mapStore.saveMapView(map.value.getZoom(), map.value.getCenter())
    })
    isReady.value = true
  })
  /**
   * 缩放地图
   * @param {number} delta - 缩放增量 (+1 或 -1)
   */
  function zoomBy(delta) {
    if (map.value) {
      map.value.setZoom(map.value.getZoom() + delta)
    }
  }
  /**
   * 将视图定位到指定像素坐标
   * @param {[number, number]} latlng - [pixelY, pixelX]
   * @param {number} zoom - 目标缩放级别
   */
  function flyTo(latlng, zoom = 14) {
    if (map.value) {
      map.value.flyTo(latlng, zoom, { duration: 0.8 })
```

```
  }
 }
 /**
  * 获取当前地图实例
  */
 function getMap() {
  return map.value
 }
 onUnmounted(() => {
  if (map.value) {
   map.value.remove()
   map.value = null
  }
 })
 return {
  map,
  isReady,
  zoomBy,
  flyTo,
  getMap,
 }
}
```

// 文件: main.js

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import router from './router'
import App from './App.vue'
// Vant 基础样式
import 'vant/lib/index.css'
// Leaflet 样式
import 'leaflet/dist/leaflet.css'
// 全局样式
import './styles/variables.css'
const app = createApp(App)
app.use(createPinia())
app.use(router)
app.mount('#app')
```

// 文件: router/index.js

```
import { createRouter, createWebHistory } from 'vue-router'
const routes = [
 {
  path: '/',
  name: 'Home',
  component: () => import('@/views/HomeView.vue'),
 },
```

```
  {
    path: '/poi/:id',
    name: 'PoiDetail',
    component: () => import('@/views/PoiDetailView.vue'),
  },
  {
    path: '/search',
    name: 'Search',
    component: () => import('@/views/SearchView.vue'),
  },
  {
    path: '/panorama',
    name: 'Panorama',
    component: () => import('@/views/PanoramaView.vue'),
  },
]
const router = createRouter({
  history: createWebHistory(),
  routes,
})
export default router
```

// 文件: stores/audio.js

```
import { defineStore } from 'pinia'
import { ref } from 'vue'
export const useAudioStore = defineStore('audio', () => {
  // 当前播放的 POI ID
  const currentPoiId = ref(null)
  // 播放状态
  const isPlaying = ref(false)
  // 当前播放进度（秒）
  const currentTime = ref(0)
  // 总时长（秒）
  const duration = ref(0)
  // Audio 实例引用
  let audioInstance = null
  /**
   * 播放指定 POI 的语音
   * @param {string} poiId
   * @param {string} url - 音频 URL
   */
  function play(poiId, url) {
    // 如果已经在播放同一个，暂停
    if (currentPoiId.value === poiId && isPlaying.value) {
      pause()
      return
```

```
  }
  // 切换到新的音频
  if (audioInstance) {
    audioInstance.pause()
    audioInstance = null
  }
  audioInstance = new Audio(url)
  currentPoiId.value = poiId
  isPlaying.value = true
  audioInstance.addEventListener('timeupdate', () => {
    currentTime.value = audioInstance.currentTime
  })
  audioInstance.addEventListener('loadedmetadata', () => {
    duration.value = audioInstance.duration
  })
  audioInstance.addEventListener('ended', () => {
    isPlaying.value = false
    currentTime.value = 0
  })
  audioInstance.play().catch(() => {
    isPlaying.value = false
  })
}
function pause() {
  if (audioInstance) {
    audioInstance.pause()
    isPlaying.value = false
  }
}
function resume() {
  if (audioInstance) {
    audioInstance.play().catch(() => {
      isPlaying.value = false
    })
    isPlaying.value = true
  }
}
function stop() {
  if (audioInstance) {
    audioInstance.pause()
    audioInstance.currentTime = 0
    audioInstance = null
  }
  currentPoiId.value = null
  isPlaying.value = false
  currentTime.value = 0
```

```
    duration.value = 0
  }
  /**
   * 跳转到指定时间
   * @param {number} time - 秒
   */
  function seek(time) {
    if (audioInstance) {
      audioInstance.currentTime = time
      currentTime.value = time
    }
  }
  return {
    currentPoild,
    isPlaying,
    currentTime,
    duration,
    play,
    pause,
    resume,
    stop,
    seek,
  }
})
```

// 文件: stores/map.js

```
import { defineStore } from 'pinia'
import { ref } from 'vue'
import { mapConfig } from '@/data/mock'
export const useMapStore = defineStore('map', () => {
  // 图片尺寸（运行时从图片加载获取）
  const imgWidth = ref(0)
  const imgHeight = ref(0)
  // Pad 后的正方形边长（用于 CRS bounds，保证 Leaflet 瓦片坐标正确）
  const paddedDim = ref(0)
  // CRS.Simple bounds: [[0,0], [paddedDim, paddedDim]]
  const bounds = ref([
    [0, 0],
    [0, 0],
  ])
  // 地图中心（像素坐标）
  const center = ref([0, 0])
  // 缩放级别
  const zoom = ref(0)
  // 上次用户操作的地图状态（用于路由返回时恢复）
  const lastZoom = ref(null)
```

```
const lastCenter = ref(null)
// 缩放范围
const minZoom = 0
const maxZoom = ref(12)
const initialZoom = ref(0)
// 瓦片路径
const tilesPath = ref('/map/tiles')
// 图片路径
const imagePath = ref('')
// 配置是否已加载
const configLoaded = ref(false)
/**
 * 加载地图配置和图片尺寸
 * 当前从 mock 数据读取，后续切换为后端 API
 */
async function loadConfig() {
  try {
    const config = mapConfig
    tilesPath.value = config.tilesPath || '/map/tiles'
    imagePath.value = config.image || '/map/hand-drawn.jpg'
    // 加载图片获取尺寸
    const img = new Image()
    img.src = config.image
    await new Promise((resolve, reject) => {
      img.onload = resolve
      img.onerror = reject
    })
    imgWidth.value = img.naturalWidth
    imgHeight.value = img.naturalHeight
    // Pad 为正方形，保证 Leaflet 瓦片坐标系统正确
    paddedDim.value = Math.max(imgWidth.value, imgHeight.value)
    // CRS bounds: [[0,0], [paddedDim, paddedDim]]
    bounds.value = [
      [0, 0],
      [paddedDim.value, paddedDim.value],
    ]
    center.value = [paddedDim.value / 2, paddedDim.value / 2]
    // 自动计算 maxZoom（基于 pad 后的正方形边长）
    if (config.maxZoom != null) {
      maxZoom.value = config.maxZoom
    } else {
      maxZoom.value = Math.ceil(Math.log2(paddedDim.value / 256))
    }
    // 初始缩放级别（可在 config.json 中配置，默认 = maxZoom - 2）
    initialZoom.value = config.initialZoom ?? Math.max(0, maxZoom.value - 2)
    // 初始 zoom
```

```
      zoom.value = initialZoom.value
      configLoaded.value = true
      console.log(`[MapStore] 配置加载完成: ${imgWidth.value}×${imgHeight.value},
paddedDim=${paddedDim.value}, maxZoom=${maxZoom.value}, initialZoom=${initialZoom.value}`)
    } catch (err) {
      console.error('[MapStore] 配置加载失败:', err)
    }
  }
  /**
   * 将百分比坐标转换为 CRS.Simple 像素坐标 [crsY, crsX]
   * @param {number} x - 百分比 X (0-100)
   * @param {number} y - 百分比 Y (0-100，0=顶部，100=底部)
   * @returns {[number, number]} [crsY, crsX]
   */
  function percentToLatLng(x, y) {
    const pixelX = (x / 100) * imgWidth.value
    const pixelY = (y / 100) * imgHeight.value
    return [pixelY, pixelX]
  }
  function pixelToPercent(crsY, crsX) {
    const x = (crsX / imgWidth.value) * 100
    const y = (crsY / imgHeight.value) * 100
    return { x: Math.round(x * 100) / 100, y: Math.round(y * 100) / 100 }
  }
  /**
   * 保存当前地图状态（缩放级别 + 中心点）
   */
  function saveMapView(zoomLevel, centerLatLng) {
    lastZoom.value = zoomLevel
    lastCenter.value = centerLatLng
  }
  return {
    imgWidth,
    imgHeight,
    paddedDim,
    bounds,
    center,
    zoom,
    minZoom,
    maxZoom,
    initialZoom,
    lastZoom,
    lastCenter,
    tilesPath,
    imagePath,
    configLoaded,
```

```
    loadConfig,
    percentToLatLng,
    pixelToPercent,
    saveMapView,
  }
})
```

// 文件: stores/poi.js

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import { mockPois } from '@/data/mock'
export const usePoiStore = defineStore('poi', () => {
  // POI 分类定义
  const categories = ref([
    { key: 'all', label: '全部', icon: 'apps-o', color: '#1989fa' },
    { key: 'scenic', label: '讲解', icon: 'flag-o', color: '#1989fa' },
    { key: 'food', label: '美食', icon: 'fire-o', color: '#ff976a' },
    { key: 'hotel', label: '住宿', icon: 'hotel-o', color: '#07c160' },
    { key: 'shopping', label: '购物', icon: 'shopping-cart-o', color: '#f44336' },
    { key: 'toilet', label: '卫生间', icon: 'underway-o', color: '#9c27b0' },
    { key: 'parking', label: '停车场', icon: 'logistics', color: '#607d8b' },
    { key: 'service', label: '游客服务', icon: 'service-o', color: '#ff5722' },
  ])
  // 全部 POI 数据
  const pois = ref(mockPois)
  // 当前选中的分类 Tab
  const activeCategory = ref('all')
  // 当前选中的 POI（点击标记后）
  const selectedPoi = ref(null)
  // 根据当前分类过滤 POI 列表
  const filteredPois = computed(() => {
    if (activeCategory.value === 'all') return pois.value
    return pois.value.filter((p) => p.category === activeCategory.value)
  })
  // 根据 ID 获取 POI
  function getPoiById(id) {
    return pois.value.find((p) => p.id === id)
  }
  // 设置当前分类
  function setCategory(key) {
    activeCategory.value = key
  }
  // 选中 POI
  function selectPoi(poi) {
    selectedPoi.value = poi
  }
```

```
  // 清除选中
  function clearSelection() {
    selectedPoi.value = null
  }
  // 搜索 POI
  function searchPois(keyword) {
    if (!keyword) return []
    const kw = keyword.toLowerCase()
    return pois.value.filter(
      (p) =>
        p.name.toLowerCase().includes(kw) ||
        p.description.toLowerCase().includes(kw) ||
        p.tags?.some((t) => t.toLowerCase().includes(kw))
    )
  }
  // 添加新 POI（仅内存）
  function addPoi(poi) {
    pois.value.push(poi)
  }
  return {
    categories,
    pois,
    activeCategory,
    selectedPoi,
    filteredPois,
    getPoiById,
    setCategory,
    selectPoi,
    clearSelection,
    searchPois,
    addPoi,
  }
})
```

// 文件: styles/variables.css

```
:root {
  /* 主题色 */
  --color-primary: #1989fa;
  --color-primary-light: #4da6ff;
  --color-primary-dark: #0d74d6;
  /* 功能色 */
  --color-success: #07c160;
  --color-warning: #ff976a;
  --color-danger: #ee0a24;
  /* POI 分类颜色 */
  --color-scenic: #1989fa;
```

```
  --color-food: #ff976a;
  --color-hotel: #07c160;
  --color-shopping: #f44336;
  --color-toilet: #9c27b0;
  --color-parking: #607d8b;
  --color-service: #ff5722;
  /* 中性色 */
  --color-text: #323233;
  --color-text-secondary: #969799;
  --color-bg: #f7f8fa;
  --color-white: #ffffff;
  --color-border: #ebedf0;
  /* 间距 */
  --spacing-xs: 4px;
  --spacing-sm: 8px;
  --spacing-md: 12px;
  --spacing-lg: 16px;
  --spacing-xl: 24px;
  /* 圆角 */
  --radius-sm: 4px;
  --radius-md: 8px;
  --radius-lg: 12px;
  --radius-round: 999px;
  /* 阴影 */
  --shadow-sm: 0 1px 4px rgba(0, 0, 0, 0.08);
  --shadow-md: 0 2px 12px rgba(0, 0, 0, 0.12);
  --shadow-lg: 0 4px 24px rgba(0, 0, 0, 0.16);
  /* 层级 */
  --z-map: 1;
  --z-overlay: 10;
  --z-header: 100;
  --z-bottom-sheet: 200;
  --z-popup: 300;
}
```

// 文件: views/HomeView.vue

```
<template>
  <div class="home-view">
    <!-- 顶部栏：搜索 + 游客计数器 -->
    <header class="home-view__header">
      <SearchBar @click="goSearch" />
      <VisitorCounter :count="425" />
    </header>
    <!-- 地图区域 -->
    <div class="home-view__map-area">
      <LeafletMap
```

```
      ref="mapRef"
      :edit-mode="editMode"
      @add-poi="onAddPoi"
    />
    <!-- 缩放 + 编辑控件 -->
    <div class="home-view__controls">
      <MapControls @zoom-in="handleZoomIn" @zoom-out="handleZoomOut" />
      <van-button
        size="small"
        :type="editMode ? 'success' : 'default'"
        @click="toggleEditMode"
      >
        {{ editMode ? '完成' : '编辑' }}
      </van-button>
    </div>
  </div>
  <!-- 语音播放条（全局悬浮） -->
  <AudioPlayer />
  <!-- POI 底部弹窗 -->
  <PoiBottomSheet />
  <!-- POI 编辑弹窗 -->
  <PoiEditDialog
    v-model:show="showEditDialog"
    :x="pendingPoi.x"
    :y="pendingPoi.y"
    @confirm="onEditConfirm"
  />
  <!-- 底部 POI 分类 Tab -->
  <CategoryTabs />
 </div>
</template>
<script setup>
import { ref } from 'vue'
import { useRouter } from 'vue-router'
import LeafletMap from '@/components/map/LeafletMap.vue'
import MapControls from '@/components/map/MapControls.vue'
import CategoryTabs from '@/components/map/CategoryTabs.vue'
import PoiEditDialog from '@/components/map/PoiEditDialog.vue'
import SearchBar from '@/components/common/SearchBar.vue'
import VisitorCounter from '@/components/common/VisitorCounter.vue'
import PoiBottomSheet from '@/components/common/PoiBottomSheet.vue'
import AudioPlayer from '@/components/common/AudioPlayer.vue'
import { usePoiStore } from '@/stores/poi'
const router = useRouter()
const mapRef = ref(null)
const poiStore = usePoiStore()
```

```
// 编辑模式
const editMode = ref(false)
const showEditDialog = ref(false)
const pendingPoi = ref({ x: 0, y: 0 })
// 从 composable 获取缩放方法（通过 mapRef 获取地图实例）
function handleZoomIn() {
  mapRef.value?.getMap()?.zoomIn()
}
function handleZoomOut() {
  mapRef.value?.getMap()?.zoomOut()
}
function goSearch() {
  router.push('/search')
}
function toggleEditMode() {
  editMode.value = !editMode.value
}
function onAddPoi({ x, y }) {
  pendingPoi.value = { x, y }
  showEditDialog.value = true
}
function onEditConfirm(poi) {
  poiStore.addPoi(poi)
  showEditDialog.value = false
}
</script>
<style scoped>
.home-view {
  display: flex;
  flex-direction: column;
  height: 100vh;
  height: 100dvh;
  overflow: hidden;
}
.home-view__header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: var(--spacing-sm);
  padding: var(--spacing-sm) var(--spacing-lg);
  padding-top: calc(var(--spacing-sm) + env(safe-area-inset-top));
  background: var(--color-white);
  z-index: var(--z-header);
  flex-shrink: 0;
}
.home-view__map-area {
```

```
  flex: 1;
  position: relative;
  min-height: 0;
}
.home-view__controls {
  position: absolute;
  right: var(--spacing-lg);
  bottom: var(--spacing-lg);
  z-index: 1000;
  display: flex;
  flex-direction: column;
  gap: var(--spacing-sm);
  align-items: flex-end;
}
</style>
```

// 文件: views/PanoramaView.vue

```
<template>
  <div class="panorama-view">
    <!-- 顶部导航栏 -->
    <div class="panorama-view__nav">
      <button class="panorama-view__back" @click="goBack">
        <van-icon name="arrow-left" size="20" />
        <span>返回</span>
      </button>
      <span class="panorama-view__title">{{ title }}</span>
      <div class="panorama-view__placeholder"></div>
    </div>
    <!-- 全景容器 -->
    <div ref="viewerRef" class="panorama-view__viewer"></div>
    <!-- 底部提示 -->
    <div v-if="showHint" class="panorama-view__hint">
      <van-icon name="scan" size="16" />
      <span>滑动屏幕旋转视角，双指缩放</span>
      <van-icon name="cross" size="14" class="panorama-view__hint-close" @click="showHint = false" />
    </div>
  </div>
</template>
<script setup>
import { ref, onMounted, onUnmounted } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import { Viewer } from '@photo-sphere-viewer/core'
import '@photo-sphere-viewer/core/index.css'
const route = useRoute()
const router = useRouter()
const viewerRef = ref(null)
```

```
const title = ref('VR 全景')
const showHint = ref(true)
let viewer = null
// 全景图片映射：POI ID → 图片路径（后续替换为真实全景图）
const PANORAMA_MAP = {
  // 示例：所有 POI 都指向 demo 全景图
  default: '/panorama/demo.jpg',
}
function getPanoramaUrl() {
  const poiId = route.query.poiId
  if (poiId && PANORAMA_MAP[poiId]) {
    return PANORAMA_MAP[poiId]
  }
  return PANORAMA_MAP.default
}
function goBack() {
  if (window.history.length > 1) {
    router.back()
  } else {
    router.push('/')
  }
}
onMounted(() => {
  const poiName = route.query.poiName
  if (poiName) {
    title.value = poiName
  }
  const url = getPanoramaUrl()
  viewer = new Viewer({
    container: viewerRef.value,
    panorama: url,
    defaultZoomLvl: 0,
    fisheye: false,
    navbar: 'zoom fullscreen',
    touchmoveTwoFingers: false, // 单指即可旋转
    mousewheel: true,
    mousewheelCtrlKey: false,
    moveInertia: true, // 惯性滑动
    minFov: 30,
    maxFov: 90,
    lang: {
      twoFingers: '请用双指滑动旋转视角',
    },
  })
  // 5 秒后自动隐藏操作提示
  setTimeout(() => {
```

```
    showHint.value = false
  }, 5000)
})
onUnmounted(() => {
  if (viewer) {
    viewer.destroy()
    viewer = null
  }
})
</script>
<style scoped>
.panorama-view {
  position: fixed;
  inset: 0;
  background: #000;
  display: flex;
  flex-direction: column;
  z-index: 9999;
}
.panorama-view__nav {
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  z-index: 10;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 12px 16px;
  padding-top: calc(12px + env(safe-area-inset-top));
  background: linear-gradient(to bottom, rgba(0, 0, 0, 0.6), transparent);
}
.panorama-view__back {
  display: flex;
  align-items: center;
  gap: 4px;
  color: white;
  background: rgba(255, 255, 255, 0.15);
  border: none;
  border-radius: 20px;
  padding: 6px 14px;
  font-size: 14px;
  cursor: pointer;
  backdrop-filter: blur(4px);
}
.panorama-view__back:active {
```

```
  background: rgba(255, 255, 255, 0.25);
}
.panorama-view__title {
  color: white;
  font-size: 16px;
  font-weight: 500;
}
.panorama-view__placeholder {
  width: 60px; /* 占位，保持标题居中 */
}
.panorama-view__viewer {
  flex: 1;
  width: 100%;
}
.panorama-view__hint {
  position: absolute;
  bottom: calc(20px + env(safe-area-inset-bottom));
  left: 50%;
  transform: translateX(-50%);
  z-index: 10;
  display: flex;
  align-items: center;
  gap: 8px;
  color: white;
  background: rgba(0, 0, 0, 0.5);
  backdrop-filter: blur(8px);
  padding: 8px 16px;
  border-radius: 20px;
  font-size: 13px;
  white-space: nowrap;
  animation: fadeInUp 0.3s ease;
}
.panorama-view__hint-close {
  cursor: pointer;
  opacity: 0.6;
}
@keyframes fadeInUp {
  from {
    opacity: 0;
    transform: translateX(-50%) translateY(10px);
  }
  to {
    opacity: 1;
    transform: translateX(-50%) translateY(0);
  }
}
```

```
</style>
```

// 文件: views/PoiDetailView.vue

```
<template>
  <div class="poi-detail-view">
    <!-- 顶部导航栏 -->
    <van-nav-bar
      title="详情"
      left-arrow
      @click-left="router.back()"
      :border="false"
      fixed
      placeholder
    />
    <!-- 可滚动内容 -->
    <div class="poi-detail-view__content">
      <!-- POI 基本信息 -->
      <PoiInfo v-if="poi" :poi="poi" />
      <!-- VR 全景入口 -->
      <VREntry v-if="poi" :poi-id="poi.id" :poi-name="poi.name" />
      <!-- 视频播放 -->
      <div v-if="poi?.videoUrl" class="poi-detail-view__section">
        <h3 class="poi-detail-view__section-title">视频介绍</h3>
        <VideoPlayer :video-url="poi.videoUrl" />
      </div>
      <!-- 语音讲解 -->
      <div v-if="poi?.audioUrl" class="poi-detail-view__section">
        <h3 class="poi-detail-view__section-title">语音讲解</h3>
        <div class="poi-detail-view__audio-card" @click="handlePlayAudio">
          <van-icon
            :name="isPlaying ? 'pause-circle' : 'play-circle'"
            size="36"
            color="var(--color-primary)"
          />
          <div class="poi-detail-view__audio-info">
            <span>{{ isPlaying ? '正在播放...' : '点击播放语音讲解' }}</span>
            <span class="poi-detail-view__audio-duration">
              {{ formatDuration(poi.audioDuration) }}
            </span>
          </div>
        </div>
      </div>
    </div>
    <!-- 底部操作栏 -->
    <ActionButtons v-if="poi" :poi="poi" />
  </div>
```

```
</template>
<script setup>
import { computed } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import { usePoiStore } from '@/stores/poi'
import { useAudio } from '@/composables/useAudio'
import PoiInfo from '@/components/detail/PoiInfo.vue'
import VREntry from '@/components/detail/VREntry.vue'
import VideoPlayer from '@/components/detail/VideoPlayer.vue'
import ActionButtons from '@/components/detail/ActionButtons.vue'
const route = useRoute()
const router = useRouter()
const poiStore = usePoiStore()
const { audioStore, playPoiAudio, formatTime } = useAudio()
const poi = computed(() => poiStore.getPoiById(route.params.id))
const isPlaying = computed(
  () => audioStore.isPlaying && audioStore.currentPoiId === poi.value?.id
)
function handlePlayAudio() {
  if (!poi.value) return
  playPoiAudio(poi.value)
}
function formatDuration(seconds) {
  return formatTime(seconds)
}
</script>
<style scoped>
.poi-detail-view {
  min-height: 100vh;
  min-height: 100dvh;
  display: flex;
  flex-direction: column;
  background: var(--color-white);
}
.poi-detail-view__content {
  flex: 1;
  overflow-y: auto;
  -webkit-overflow-scrolling: touch;
}
.poi-detail-view__section {
  padding: var(--spacing-lg);
  padding-top: 0;
}
.poi-detail-view__section-title {
  font-size: 16px;
  font-weight: 600;
```

```
  color: var(--color-text);
  margin: 0 0 var(--spacing-md);
}
.poi-detail-view__audio-card {
  display: flex;
  align-items: center;
  gap: var(--spacing-md);
  padding: var(--spacing-lg);
  background: var(--color-bg);
  border-radius: var(--radius-lg);
  cursor: pointer;
}
.poi-detail-view__audio-card:active {
  background: var(--color-border);
}
.poi-detail-view__audio-info {
  display: flex;
  flex-direction: column;
  gap: 2px;
  font-size: 14px;
  color: var(--color-text);
}
.poi-detail-view__audio-duration {
  font-size: 12px;
  color: var(--color-text-secondary);
}
</style>
```

// 文件: views/SearchView.vue

```
<template>
  <div class="search-view">
    <!-- 搜索栏 -->
    <van-nav-bar :border="false" fixed placeholder>
      <template #left>
        <van-icon name="arrow-left" size="20" @click="router.back()" />
      </template>
      <template #title>
        <div class="search-view__input-wrap">
          <van-icon name="search" />
          <input
            ref="inputRef"
            v-model="keyword"
            class="search-view__input"
            type="search"
            placeholder="搜索景点、美食、住宿..."
            @input="handleSearch"
```

```
        />
        <van-icon v-if="keyword" name="clear" class="search-view__clear" @click="clearSearch" />
      </div>
    </template>
  </van-nav-bar>
  <!-- 搜索结果 -->
  <div class="search-view__results">
    <!-- 默认状态 -->
    <div v-if="!keyword" class="search-view__empty">
      <van-icon name="search" size="48" color="#ddd" />
      <p>输入关键词搜索</p>
    </div>
    <!-- 无结果 -->
    <div v-else-if="!results.length" class="search-view__empty">
      <van-icon name="warning-o" size="48" color="#ddd" />
      <p>未找到相关结果</p>
    </div>
    <!-- 结果列表 -->
    <div v-else class="search-view__list">
      <div
        v-for="poi in results"
        :key="poi.id"
        class="search-view__item"
        @click="goDetail(poi)"
      >
        <div class="search-view__item-icon" :style="{ background: getCategoryColor(poi.category) }">
          <van-icon :name="getCategoryIcon(poi.category)" color="white" size="14" />
        </div>
        <div class="search-view__item-info">
          <h4 class="search-view__item-name">{{ poi.name }}</h4>
          <p class="search-view__item-desc">{{ poi.description }}</p>
          <div v-if="poi.tags?.length" class="search-view__item-tags">
            <van-tag v-for="tag in poi.tags.slice(0, 2)" :key="tag" plain size="medium">
              {{ tag }}
            </van-tag>
          </div>
        </div>
        <van-icon name="arrow" class="search-view__item-arrow" />
      </div>
    </div>
  </div>
</template>
<script setup>
import { ref, onMounted } from 'vue'
import { useRouter } from 'vue-router'
```

```
import { usePoiStore } from '@/stores/poi'
const router = useRouter()
const poiStore = usePoiStore()
const inputRef = ref(null)
const keyword = ref('')
const results = ref([])
onMounted(() => {
  inputRef.value?.focus()
})
function handleSearch() {
  results.value = poiStore.searchPois(keyword.value)
}
function clearSearch() {
  keyword.value = ''
  results.value = []
}
function goDetail(poi) {
  router.push(`/poi/${poi.id}`)
}
function getCategoryColor(category) {
  return poiStore.categories.find((c) => c.key === category)?.color || '#1989fa'
}
function getCategoryIcon(category) {
  return poiStore.categories.find((c) => c.key === category)?.icon || 'flag-o'
}
</script>
<style scoped>
.search-view {
  min-height: 100vh;
  min-height: 100dvh;
  background: var(--color-bg);
}
.search-view__input-wrap {
  display: flex;
  align-items: center;
  gap: var(--spacing-sm);
  background: var(--color-bg);
  border-radius: var(--radius-round);
  padding: var(--spacing-xs) var(--spacing-md);
  flex: 1;
  min-width: 0; /* 防止 flex 子项溢出 */
}
/* 覆盖 Vant NavBar title 的 van-ellipsis 裁剪 */
.search-view :deep(.van-nav-bar__title) {
  overflow: visible;
  max-width: none;
```

```
}
.search-view__input {
  flex: 1;
  min-width: 0; /* flex 子项必须设 min-width:0 才能正确收缩 */
  border: none;
  background: transparent;
  font-size: 14px;
  outline: none;
  color: var(--color-text);
}
/* 隐藏浏览器原生 search 清除按钮，使用自定义 van-icon */
.search-view__input::-webkit-search-cancel-button {
  display: none;
}
.search-view__input::placeholder {
  color: var(--color-text-secondary);
}
.search-view__clear {
  color: var(--color-text-secondary);
  cursor: pointer;
  flex-shrink: 0;
}
.search-view__results {
  padding: var(--spacing-lg);
}
.search-view__empty {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: var(--spacing-md);
  padding: 60px 0;
  color: var(--color-text-secondary);
  font-size: 14px;
}
.search-view__list {
  display: flex;
  flex-direction: column;
  gap: var(--spacing-md);
}
.search-view__item {
  display: flex;
  align-items: center;
  gap: var(--spacing-md);
  background: var(--color-white);
  border-radius: var(--radius-lg);
  padding: var(--spacing-md) var(--spacing-lg);
```

```
  cursor: pointer;
  box-shadow: var(--shadow-sm);
}
.search-view__item:active {
  background: var(--color-bg);
}
.search-view__item-icon {
  width: 36px;
  height: 36px;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
}
.search-view__item-info {
  flex: 1;
  min-width: 0;
}
.search-view__item-name {
  font-size: 15px;
  font-weight: 500;
  color: var(--color-text);
  margin: 0 0 2px;
}
.search-view__item-desc {
  font-size: 12px;
  color: var(--color-text-secondary);
  margin: 0 0 4px;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
.search-view__item-tags {
  display: flex;
  gap: var(--spacing-xs);
}
.search-view__item-arrow {
  color: var(--color-text-secondary);
  flex-shrink: 0;
}
</style>
```

// 文件: scripts/generate-tiles-leaflet.js

```
/**
 * 手绘地图瓦片生成脚本（CRS.Simple 像素坐标系）
```

```
 * 用法: node scripts/generate-tiles-leaflet.js
 *
 * 原理:
 * - 使用 Leaflet 标准 CRS.Simple 像素坐标系（带 y-flip）
 * - 将图片 pad 为正方形后按标准瓦片金字塔切割
 * - 瓦片 y 轴翻转：文件 ty=0 对应图像底部（匹配 CRS.Simple 的 y-flip 约定）
 * - 保证每片瓦片内容区域与 Leaflet 定位区域一致，零形变
 * - 配置驱动：读取 src/data/mock.js 中的 mapConfig 获取图片路径和输出路径
 */
import sharp from 'sharp'
import { mkdirSync, existsSync, rmSync } from 'node:fs'
import { join, dirname } from 'node:path'
import { fileURLToPath } from 'node:url'
const __dirname = dirname(fileURLToPath(import.meta.url))
const ROOT = join(__dirname, '..')
// ===== 读取配置（从 mock 数据）=====
const { mapConfig } = await import(join(ROOT, 'src/data/mock.js'))
const config = mapConfig
const SOURCE = join(ROOT, 'public', config.image.replace(/^\//, ''))
const OUTPUT = join(ROOT, 'public', config.tilesPath.replace(/^\//, ''))
const TILE_SIZE = 256
async function generateTiles() {
  const metadata = await sharp(SOURCE).metadata()
  const imgW = metadata.width
  const imgH = metadata.height
  // Pad 为正方形，边长 = max(imgW, imgH)
  const paddedDim = Math.max(imgW, imgH)
  // 计算最大缩放级别：使 maxZoom 级每片瓦片约 256px
  const maxZoom = config.maxZoom ?? Math.ceil(Math.log2(paddedDim / TILE_SIZE))
  console.log(`📷 源图片: ${imgW} × ${imgH}`)
  console.log(`📐 填充为: ${paddedDim} × ${paddedDim}`)
  console.log(`🔍 最大缩放: z=${maxZoom}（每片约 ${Math.round(paddedDim / Math.pow(2, maxZoom))}×
${Math.round(paddedDim / Math.pow(2, maxZoom))}px）`)
  console.log(`📁 输出目录: ${OUTPUT}`)
  console.log('---')
  // 清理旧瓦片
  if (existsSync(OUTPUT)) {
    rmSync(OUTPUT, { recursive: true })
    console.log('🗑 已清理旧瓦片目录')
  }
  // 将图片 pad 为正方形（图片在左上角，右下角填充透明）
  const paddedBuffer = await sharp(SOURCE)
    .resize(paddedDim, paddedDim, { fit: 'contain', position: 'left top', background: { r: 0, g: 0, b: 0, alpha: 0 } })
    .png()
    .toBuffer()
```

```
  let totalTiles = 0
  for (let z = 0; z <= maxZoom; z++) {
    // 标准瓦片金字塔：z 级网格 = 2^z × 2^z
    const tilesPerSide = Math.pow(2, z)
    const tileSizeInPadded = paddedDim / tilesPerSide // 每片瓦片在 pad 正方形中的像素尺寸
    let levelTiles = 0
    for (let ty = 0; ty < tilesPerSide; ty++) {
      for (let tx = 0; tx < tilesPerSide; tx++) {
        // CRS.Simple 的 y-flip 约定：ty=0 → 图像底部，ty=max → 图像顶部
        // 文件 ty 对应 pad 正方形中的行（从底部开始）
        const fileRow = (tilesPerSide - 1 - ty) // 翻转 y：文件行从顶部开始
        // 瓦片在 pad 正方形中的像素区域（从顶部计算）
        const left = Math.round(tx * tileSizeInPadded)
        const top = Math.round(fileRow * tileSizeInPadded)
        const width = Math.round((tx + 1) * tileSizeInPadded) - left
        const height = Math.round((fileRow + 1) * tileSizeInPadded) - top
        // 创建目录和瓦片路径
        const tileDir = join(OUTPUT, String(z), String(tx))
        if (!existsSync(tileDir)) {
          mkdirSync(tileDir, { recursive: true })
        }
        const tilePath = join(tileDir, `${ty}.png`)
        // 检查瓦片是否与原图有交集
        // 原图占据 pad 正方形的左上角: [0,0] ~ [imgW, imgH]
        const overlapLeft = Math.max(left, 0)
        const overlapTop = Math.max(top, 0)
        const overlapRight = Math.min(left + width, imgW)
        const overlapBottom = Math.min(top + height, imgH)
        if (overlapRight <= overlapLeft || overlapBottom <= overlapTop) {
          // 瓦片完全在图片区域外，生成透明瓦片（避免 Leaflet 显示灰色）
          await sharp({ create: { width: TILE_SIZE, height: TILE_SIZE, channels: 4, background: { r: 0, g: 0, b: 0, alpha:
0 } } })
            .png()
            .toFile(tilePath)
          totalTiles++
          levelTiles++
          continue
        }
        await sharp(paddedBuffer)
          .extract({ left, top, width, height })
          .resize(TILE_SIZE, TILE_SIZE, { fit: 'fill' })
          .png()
          .toFile(tilePath)
        totalTiles++
        levelTiles++
      }
```

```
    }
    console.log(`z=${z}: ${tilesPerSide}×${tilesPerSide} 网格, 有效 ${levelTiles} 片`)
  }
  console.log('---')
  console.log(`✅ 完成！ 共生成 ${totalTiles} 个瓦片`)
  console.log(`📁 输出目录: ${OUTPUT}`)
  console.log(`💡 maxNativeZoom: ${maxZoom}`)
  console.log(`💡 paddedDim: ${paddedDim}（前端需使用此值设置 CRS bounds）`)
}
generateTiles().catch((err) => {
  console.error('❌ 生成失败:', err.message)
  process.exit(1)
})
```